Self-Evolving Recommendation System: End-To-End Autonomous Model Optimization With LLM Agents

Haochen Wang*
Google Inc
Mountain View, California, USA
haochenww@google.com

Yi Wu*
Google Inc
Mountain View, California, USA
wuyish@google.com

Daryl Chang*
Google Inc
Mountain View, California, USA
dlchang@google.com

Li Wei
Google Inc
Mountain View, California, USA
liwei@google.com

Lukasz Heldt
Google Inc
Mountain View, California, USA
heldt@google.com

Abstract

Optimizing large-scale machine learning systems, such as recommendation models for global video platforms, requires navigating a massive hyperparameter search space and, more critically, designing sophisticated optimizers, architectures, and reward functions to capture nuanced user behaviors. Achieving substantial improvements in these areas is a non-trivial task, traditionally relying on extensive manual iterations to test new hypotheses. We propose a self-evolving system that leverages Large Language Models (LLMs), specifically those from Google's Gemini family, to autonomously generate, train, and deploy high-performing, complex model changes within an end-to-end automated workflow. The self-evolving system is comprised of an Offline Agent (Inner Loop) that performs high-throughput hypothesis generation using proxy metrics, and an Online Agent (Outer Loop) that validates candidates against delayed north star business metrics in live production. Our agents act as specialized Machine Learning Engineers (MLEs): they exhibit deep reasoning capabilities, discovering novel improvements in optimization algorithms and model architecture, and formulating innovative reward functions that target long-term user engagement. The effectiveness of this approach is demonstrated through several successful production launches at YouTube, confirming that autonomous, LLM-driven evolution can surpass traditional engineering workflows in both development velocity and model performance.

CCS Concepts

• **Information systems** → **Recommender systems**; **Language models**.

Keywords

Large Language Model, Autonomous Agent, Recommendation System

1 Introduction

Global video platforms like YouTube serve billions of users by curating personalized feeds from vast corpora of content. At the core of delivering relevant experiences is the recommendation system, an ensemble of algorithms and models designed to help users discover content they love. Increasingly, modern recommendation systems are being formulated as Reinforcement Learning (RL) problems [4, 33], where the system acts as an agent interacting with a user environment to maximize cumulative utility over time. As surveyed [1], this paradigm shifts the focus from simple Click-Through Rate (CTR) prediction to optimizing long-term user satisfaction, requiring models to balance immediate gratification with delayed rewards like retention and diverse content exploration.

However, a critical bottleneck in this paradigm is the alignment gap between training proxies and long-term user satisfaction. While models are trained on differentiable loss functions, the actual goal is user satisfaction, which is non-differentiable, delayed, sparse, and often semantically complex. Recent approaches like the Learned Ranking Function [30] attempt to bridge this gap by parameterizing the reward function itself, allowing the system to learn the optimal trade-off between conflicting objectives. Similarly, work on diversifying by intent [28] highlights that modern reward functions must now encode nuanced psychological concepts – such as user intent and exploration – rather than simple binary labels.

Optimizing these increasingly semantic and structural components exceeds the capabilities of traditional Automated Machine Learning (AutoML) [35]. Standard AutoML methods [7, 25] excel at tuning numerical hyperparameters within fixed search spaces. Yet, they lack the reasoning capabilities to invent new reward logic or architect novel interaction layers from scratch. They cannot interpret past experiment results, hypothesize that a specific user slice is under-served, and write the logic to fix it.

This limitation has catalyzed a shift in the broader machine learning community from "automated tuning" to "autonomous scientific discovery." Recent work such as [15, 19] introduces the concept of AI agents capable of orchestrating the full scientific lifecycle: generating hypotheses, writing code, and refining theories based on empirical results. Unlike rigid AutoML pipelines, these agents utilize Large Language Models (LLMs) to reason over unstructured context. This capability offers a potential solution to the limitation of traditional methods, promising a transition from mere parameter tuning and selection, to automated discovery of complex, novel model changes.

Despite these parallel advancements, a significant gap remains at their intersection. Optimizing industrial-scale recommendation models proves exceptionally difficult and remains a manual, human-intense endeavor. We argue that the complexity of modern RecSys is the ideal testbed for autonomous agents, and without them, the

*Equal contribution to the work.

most high-leverage optimizations are entirely dependent on human intuition. Specifically, we identify three challenges that necessitate an agentic approach in industrial recommendation systems:

- **C1: The Intractability of Structural Design** While numerical parameters like learning rate are easily tuned via Bayesian optimization [25], the primary drivers of innovation in modern recommendation are structural mutations. At YouTube's scale, systems are typically built as deep neural networks [5, 6, 18, 34] with complex architectures and optimizers where the search space is virtually infinite. This involves discrete design choices – such as introducing activation functions (e.g., Swish [20], GELU [11]) or interaction layers (e.g., DCN [27], Transformers [13]). Standard AutoML fails here because it lacks the reasoning capabilities to navigate an open-ended design space, leaving these high-leverage optimizations entirely to human intuition.
- **C2: The Semantic Gap in Reward Engineering** The reward is the most critical component of an RL-based recommendation model, and in industrial settings, it requires extensive refinement and is rarely a static label. It is a composite logic aggregating disparate signals – watch time, survey responses, retention metrics, and more – to approximate long-term user satisfaction. Designing this function so that it captures user delight while respecting business intent demands a deep, semantic understanding of user-system interactions. This is a reasoning task that gradient-based search is not capable of. It is a formidable challenge to find the optimal trade-off in this high-dimensional reward space, as it often involves balancing conflicting signals, as well as identifying and integrating entirely new signals.
- **C3: The Scalability Limit of Human-Driven Iteration** Despite the availability of massive computational resources, the velocity of model improvement is strictly bound by human bandwidth. In traditional workflows, every experimental iteration requires significant manual effort: engineers must translate high-level hypotheses into code, configure model trainers, set up live A/B testing, and evaluate results. This human-driven workflow is inherently unscalable; the number of valid hypotheses a team can explore remains linear to the number of engineers. This bottleneck leaves vast regions of the potential solution space unexplored simply because the manual cost of implementing and monitoring each hypothesis is too high.

To bridge the gap, we introduce a **Self-Evolving Recommendation System** deployed at YouTube. By integrating recent advancements in LLMs with an industrial recommendation system, we cross a new frontier by being the first to build a rigorous framework where agents act as expert Machine Learning Engineers (MLEs) to solve global-scale open-ended recommendation modeling problems. These agents do not just tune parameters; they read production code, propose structural changes to neural topologies, and formulate complex logic for reward functions within an end-to-end autonomous pipeline. While our primary deployment utilizes Gemini 2.5 Pro [9], we evaluate the effectiveness of the lightweight Gemini variant in our ablation studies (Section 5.4.1) to quantify the relationship between model reasoning power and discovery performance.

Our contributions are summarized as follows:

- **Autonomous MLE Framework for Industrial-Scale Systems** We introduce a hierarchical agentic system where specialized LLM agents act as expert MLEs to evolve recommendation models. We detail the system design that enables agents to safely manage the full lifecycle of industrial model development – from hypothesis generation and code implementation to A/B testing.
- **Semantic Discovery** We demonstrate that LLM-based agents can move beyond simple parameter tuning to discover novel architectural components and multi-objective reward functions that align better with long-term user satisfaction, areas previously accessible only to human experts in the recommendation domain.
- **Acceleration of Experimental Velocity** We confirm the success of an autonomous LLM-based evolutionary recommendation system in accelerating the velocity of experimentation and delivery of measurable metric gains. With extensive offline and online experiments and production deployments, we validate that agentic systems can surpass hand-tuned baselines and effectively evolve the state-of-the-art in recommendation systems at YouTube.

2 Related Work

Our work sits at the intersection of automated machine learning, autonomous agents, and RL for recommendation. We distinguish our contribution by contrasting it with existing paradigms in these areas.

2.1 Automated Model Optimization

Industrial standard practices for automated model optimization in recommendation rely heavily on Hyperparameter Optimization (HPO) [2]. Frameworks like Google Vizier [10] and Auto-Sklearn [8] utilize iterative search methods (e.g., Bayesian optimization [25], Gaussian processes [21]) to tune continuous hyperparameters. However, these methods are confined to defined parameter ranges and lack the semantic depth to interpret why a configuration succeeds or fails.

Similarly, Neural Architecture Search (NAS) [7] applies these principles to successfully optimize feature interactions and embeddings. Yet, standard NAS approaches (e.g., DARTS [14], evolutionary search [22]) remain restricted by their search space definition – they can only select from a predefined menu of operations. They cannot invent novel modules, refactor code to fix bottlenecks, or introduce complex logic that was not explicitly programmed into the search space.

2.2 LLMs and Autonomous Agents for Scientific Discovery

The emergence of LLMs has enabled a shift from selection to generation. Optimization by PROmpting (OPRO) [31] demonstrates that LLMs can serve as evolutionary operators, iteratively refining solutions based on natural language descriptions. This capability is amplified by models such as Gemini 2.5 [9] with advanced reasoning capabilities, building on the foundations of Chain-of-Thought [29] and long-context understanding and thinking.

Beyond optimization, the field of AI agents has exploded with frameworks like ReAct (Reasoning + Acting) [32] and Toolformer [23], which demonstrate that LLMs can solve complex tasks by interleaving reasoning traces with external tool execution. Building on this, recent "scientist" agents have attempted to automate open-ended workflows. Voyager [26] and MetaGPT [12] introduce agents that write executable code to solve open-ended problems, maintaining a persistent repository of reusable functions to accelerate future tasks. AlphaEvolve [19], The AI Scientist [15], and MLE-STAR [17] extend this to algorithmic discovery, where agents perform direct edits on source code to improve performance on academic benchmarks. Our work adapts this "scientist" paradigm to the industrial recommendation system setting. Unlike prior works that optimize for academic benchmarks (e.g., Kaggle), our framework addresses the unique challenges of a live production ecosystem: noisy feedback loops, strict safety guardrails, complex user-system interactions, and the need for rigorous A/B testing protocols.

2.3 Reward Engineering for Reinforcement Learning

In RL, designing the reward function is often the hardest part of the problem. Eureka [16] pioneers the use of LLMs for evolutionary reward design in robotics, while LEARN-Opt [3] optimizes rewards without predefined metrics by using LLMs as analysts to evaluate candidates. However, critical distinctions remain: the availability of a clear oracle for success, and the latency of the feedback loop. While prior approaches are evaluated on robotics or simulations that offer an immediate feedback, recommendation systems lack a clear oracle for user satisfaction. The true objective is a latent variable observable only through the delayed, noisy, and sparse real-world interactions on the order of $\Theta(days)$ or $\Theta(weeks)$. Consequently, when designing a reward, we cannot simply optimize for a simulation score; we must instead reason about alignment with offline proxy signals analyzed over petabytes of interaction logs, and ultimately validate our designs through real-world deployment.

3 Problem Formulation

We begin by introducing the components of an RL-based recommendation model, specifically for the task of ranking videos to maximize the total expected long-term user satisfaction. We formulate this task as a bi-level optimization problem.

Our ultimate goal is to maximize a non-differentiable, long-term user satisfaction metric, which is observable only through online interaction. However, directly optimizing online metrics is intractable because feedback is sparse, delayed, and noisy. Therefore, the problem is bi-level: in the lower level, a ranking model is trained to optimize an engineered proxy objective (the cumulative reward). In the upper level, we find the optimal system configuration (e.g., optimizer, architecture, reward) such that the ranking model's induced policy maximizes the online metrics upon deployment.

3.1 The Lower Level: Model Training

We consider a standard recommendation setting where a ranking model, parameterized by weights θ, ranks a list of candidate items to generate an action a (the ranking order) given state s, which comprises the user state and candidate videos, to maximize a cumulative reward.

While the methodology proposed in this paper is model-agnostic, our deployment environment utilizes a value-based RL approach [30, 33]. Specifically, the model optimizes a state-action value function $Q_\theta(s, a)$ that estimates the long-term value of a ranking action, defined by a proxy reward constructed from session-level user-item interactions. The model is trained via Stochastic Gradient Descent (SGD) to minimize a differentiable proxy loss function $\mathcal{L}_{\text{proxy}}$:

$$\theta^*(\Phi) = \arg\min_\theta \mathcal{L}_{\text{proxy}}(\mathcal{D}; \theta, \Phi)$$

where $\mathcal{D}$ represents the training data logs and Φ represents the meta-configuration of the system, such as the model optimizer, architecture, and reward definition. This lower-level optimization is performed by the model trainer.

3.2 The Upper Level: Optimizing North Star Metrics

While the ranking model optimizes the proxy reward, industrial recommendation systems ultimately care about the true north star metrics $\mathcal{M}$. The mapping between the proxy reward and the true online metrics is not guaranteed; a model might improve offline loss on a poorly defined reward function to the detriment of user satisfaction.

Thus, we formulate the problem of finding the optimal configuration Φ as:

$$\Phi^* = \arg\max_\Phi \mathbb{E}\left[\mathcal{M}(\theta^*(\Phi))\right] \quad \text{s.t.} \quad \mathcal{G}(\Phi) \le C$$

Here, $\theta^*(\Phi)$ represents the model weights trained under configuration Φ, and $\mathcal{G}$ represents system-level constraints (e.g., training cost).

This formulation highlights the challenge: we must optimize Φ using expensive, noisy feedback from $\mathcal{M}$ to ensure the ranking model, which efficiently optimizes reward, is actually solving the business problem. Traditionally, optimizing Φ has been undertaken by human researchers. Our goal is to automate the role of a human researcher via an MLE agent that iteratively refines the components of Φ. Concrete examples of Φ include:

- **Optimizer ($\eta \in \Phi$)** The learning rate and update rules (e.g., AdaGrad) used to train the model weights θ.
- **Architecture ($\phi \in \Phi$)** The structure (e.g., DCN) of the ranking network.
- **Reward Definition ($r \in \Phi$)** The logic determining the training labels, combining various engagement and user signals to balance competing objectives.

4 The Self-Evolving System

We propose a system that automates the discovery of an optimal model configuration Φ^* by decoupling the discovery process into two distinct, synchronized feedback loops. The system is designed around a shared context – a persistent knowledge base containing historical offline analytics and online experiment results – which informs two primary agents (Figure 1):

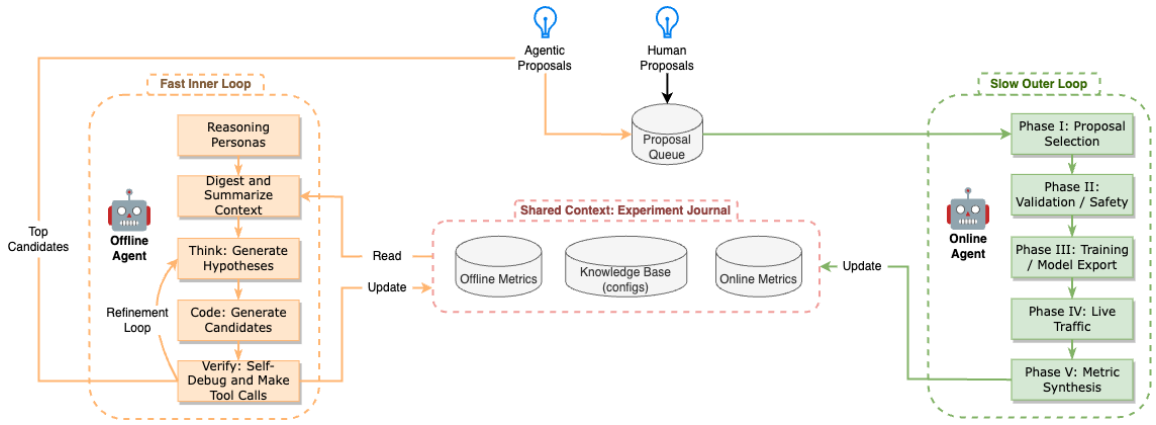

Figure 1: The Self-Evolving System Architecture. The framework operates as a dual-loop, self-evolving system centered around a shared context containing a persistent knowledge base and an Experiment Journal of historical trials and their resulting metrics. The Offline Agent (Inner Loop) serves as the high-frequency cognitive core, where LLMs are invoked to instantiate specialized reasoning personas that generate and refine hypotheses into executable code through a closed-loop "Think-Code-Verify" cycle. Offline tool calls are made to evaluate candidates and filter them to a smaller subset. High-potential survivors are promoted to the Online Agent (Outer Loop), which manages the asynchronous transition of every proposal through a five-phase Directed Acyclic Graph (DAG). This Outer Loop ensures model integrity and safety through automated push evaluations and live traffic monitoring before closing the loop by serializing online north star metrics back into the Experiment Journal.

(1) The Offline Agent (Inner Loop): Operates at high frequency to generate model improvements on a recurring basis, continuously identifying opportunities and pruning candidates down to the most promising ones using offline signals.
(2) The Online Agent (Outer Loop): Operates at low frequency to validate surviving candidates against north star metrics via live traffic.

By employing a dual-agent system instead of a monolithic agent, we establish a rigorous filtration funnel, ensuring that expensive online traffic is reserved for candidates that have already demonstrated statistical promise via cheap offline proxies.

4.1 The Offline Agent: A Fast Inner Loop

The Offline Agent serves as the cognitive core and the rapid test engine of the framework. Its primary objective is to traverse the semantic configuration space and identify high-potential candidates before they reach the expensive Outer Loop. This agent optimizes for speed and throughput, utilizing LLMs and a set of offline tools to bridge the gap between abstract research ideas and executable, high-quality candidates.

4.1.1 Prompt Construction. To ground the agent's creative agency within the rigorous constraints of a production environment, the agent prompt (see template in Appendix A) is dynamically constructed and includes:

- **Persona Framing** The agent is told it is an expert MLE with excellent programming and analytical skills. It is also given a specialized persona depending on the task.
- **Primary Objectives** The agent is tasked with proposing model changes while prioritizing key metrics. To avoid the bias toward incremental tweaks, we instruct the agent to balance exploration and exploitation across its proposals.
- **Steering Instructions** While the system is end-to-end autonomous, it includes a mechanism for human-in-the-loop steering. Engineers can provide optional instructions to direct the LLM toward specific research areas or enforce additional constraints based on recent production insights.
- **Safety Guardrails** The agent is governed by explicit instructions to enforce strict safety thresholds. For example, the agent is directed to ensure that regressions in "Metric#3" do not exceed +1%, preventing reward hacking or catastrophic drift in production.
- **Baseline Configurations and Schema** The agent is provided with the model configuration and schema (such as training logs) depending on the specialized persona.
- **Experiment Journal** This is a structured historical record of past iterations, including the specific code changes (represented as diffs) and their resulting offline and online outcomes. This allows the LLM to learn from past failures.

4.1.2 Specialized Reasoning Personas and Tool Calls. We decompose the multifaceted task of recommendation system design into specialized agent personas, each optimized for distinct modeling tasks. A single persona that is exposed to the full breadth of the codebase quickly becomes polluted with irrelevant schema definitions and knowledge, leading to "context rot" and hallucination. While each persona acts to filter the infinite possibilities of candidates down to the top candidates, the specific definition of success and the tools employed differ based on the modeling task. Therefore, we instantiate distinct specialized personas, each equipped with specific tools and objectives.

A. The Optimizer Persona (Tool: `compute_loss`) This persona searches the space of optimization algorithms. It iteratively

proposes changes to the optimizer class (e.g., Adagrad, RMSprop) and its internal hyperparameters (e.g., momentum, batch size).

For such changes, the definition of the loss function $\mathcal{L}_{\text{proxy}}$ remains invariant. While the model changes, the yardstick measuring its performance does not. Consequently, the resulting validation loss values are comparable. A lower loss implies a better approximation of the ground truth labels.

- **Objective** Minimize offline loss $\mathcal{L}_{\text{proxy}}$.
- **Tooling** The persona utilizes the `compute_loss` tool. This enables a direct sorting of candidates: $\Phi_A \succ \Phi_B \iff \mathcal{L}_{\text{proxy}}(\Phi_A) < \mathcal{L}_{\text{proxy}}(\Phi_B)$.
- **Process** The persona generates multiple configuration diffs (e.g., changing learning rates, replacing layers) and launches asynchronous training jobs.
- **Selection** The persona computes offline loss for each trainer and sorts the trainers by their loss values. Only configurations that show statistically significant improvements in offline convergence are promoted to the Online Agent.

B. The Architecture Persona (Tool: `compute_loss`) This persona specializes in the neural topology of the ranking network. It parses Keras/TensorFlow definitions of the model and proposes structural mutations. Unlike standard NAS, which selects from a fixed menu of layers, this persona can write novel code – for example, replacing standard embedding lookups with a custom "Gating Path" mechanism or introducing layer normalization in specific sub-towers (discussed in Section 5.2.3). Like the Optimizer persona, it relies on the `compute_loss` tool to verify that the new topology is learnable and more expressive than the baseline.

C. The Reward Persona (Tool: `run_sql_query`) This persona performs reward engineering by editing the logic that defines the ground-truth training label for the ranking model. Because modifying the reward fundamentally alters the optimization landscape itself, comparing $\mathcal{L}_{\text{proxy}}$ across different reward definitions is ill-defined. A model trained on a "click-only" reward will naturally have a lower loss than one trained on a complex "click + satisfaction" reward, simply because the latter task is harder to learn. Therefore, this persona cannot use `compute_loss`. Instead, it uses the `run_sql_query` tool to execute massive-scale data and correlation analyses. It validates hypotheses by confirming that the proposed reward signal correlates with desirable user behaviors (e.g., "long dwell time") in the training logs.

- **Objective** Identify promising signals via data and correlation analysis.
- **Tooling** The persona utilizes the `run_sql_query` tool. This tool validates the semantic quality of the new label via feature and correlation analysis.
- **Process** Analogous to the parallel training jobs above, the agent executes batches of asynchronous analytical queries over the training logs.
- **Selection** The goal is not to minimize a loss function, but to perform massive-scale signal discovery, identifying features or interactions that exhibit high correlation with user engagement metrics. This step allows the persona to validate the semantic richness of a potential reward term before it is ever coded into a trainer.

4.1.3 Reasoning and Code Generation Loop. The agent operates in a closed-loop "Think-Code-Verify" cycle to ensure all proposed changes are valid and deployable, adhering to the output format specified in the prompt template:

(1) *Hypothesis Generation* The agent first formulates a strategy (e.g., "The model is overfitting to clickbait; let's penalize short-duration clicks in the reward formula").
(2) *Code Implementation* It translates the hypothesis into a precise code or configuration diff.
(3) *Refinement by LLM* The generated code is passed to a "linter" persona which critically reviews the code and fixes syntax errors, ensuring compliance with the provided schema.
(4) *Tool Calls* The agent invokes the appropriate tool (from the toolkit that includes `compute_loss` and `run_sql_query`) to quantify the quality of candidates.

Only candidates that pass this rigorous Inner Loop – showing promise via their respective offline measurements – are promoted to the Outer Loop.

4.2 The Online Agent: A Slow Outer Loop

While the Offline Agent prioritizes high-throughput hypothesis generation and candidate recommendation, the Online Agent prioritizes the accuracy and safety required to push these changes to production. Its role is to validate the candidates provided by the Offline Agent against the north star business metrics $\mathcal{M}$ which are often delayed and cannot be perfectly proxied offline. It operates as a persistent orchestration service that manages the asynchronous lifecycle of every candidate model.

To handle the complexity of coordinating distributed training and live traffic diversion, the Online Agent maintains a robust state machine for every proposal Φ. This state machine transitions configurations through a Directed Acyclic Graph (DAG) of five distinct phases, ensuring that only fully validated models affect live user traffic.

4.2.1 Phase I: Proposal Selection. The lifecycle begins in the **PROPOSED** state. This phase acts as a universal sink, decoupling the source of the hypothesis from its execution. The Online Agent maintains a queue that accepts configuration manifests from heterogeneous sources:

- **Agent-Generated Candidates** The top survivors from the Offline Agent (e.g., architecture changes sorted by offline loss, or reward functions sorted by signal correlation).
- **Human-Generated Candidates** Manual overrides or baseline configurations submitted by MLEs.

By standardizing the input format, the Online Agent treats all proposals agnostically and creates a unified pipeline for both autonomous and manual proposals. Proposals are processed in a first-in, first-out (FIFO) manner by default to ensure fair resource allocation. However, the system allows for manual reordering of the queue, enabling engineers to prioritize specific trials or baseline validations for immediate execution.

4.2.2 Phase II: Model Validation. Before allocating computational resources, the system transitions to the **VALIDATED** state. Here, the agent acts as a gatekeeper, performing static analysis to ensure the configuration is deployable. This includes:

- **Compilation Check** Ensuring the model configuration can be parsed and compiled in the correct format.
- **Model Push Evaluation** Assessing the model against baseline thresholds to ensure a bad model is not pushed. Examples include checking whether there is sufficient volume of data for the model to learn effectively and performing pairwise evaluations to ensure the model has not drifted.

Failures at this stage trigger an immediate "Fast Fail," reporting an error to the user without incurring a training cost.

4.2.3 Phase III: Model Training. Valid configurations move to the **TRAINING** state. Crucially, this phase ensures availability of model.

- **Monitoring Model Training** The agent polls the model inference server, monitoring for availability of models.

The state transition strictly requires that the model weights are successfully exported, versioned, and pushed before proceeding.

4.2.4 Phase IV: Live Experimentation and Safety. The most critical phase is the **LIVE** state, where the model is exposed to production traffic. The Online Agent performs two distinct functions here:

- **Traffic Diversion** The agent interacts with the experiment server to allocate a statistically significant slice of traffic to the new model arm.
- **Duration Management and Safety Guardrails** The experiment is maintained for a specific duration to capture delayed metrics. During this window, the agent continuously monitors online metrics. If any metric violates a threshold, the agent can abort the experiment to protect the user experience.

4.2.5 Phase V: Metric Synthesis and Loop Closure. Finally, the lifecycle concludes in the **COMPLETED** state. This is not merely a termination status but an active data ingestion phase.

- **North Star Retrieval** The agent queries the experimentation server to retrieve the final list of online metrics $\mathcal{M}$ (e.g., watch time).
- **Context Synchronization** These metrics and statuses are serialized and written into the Experiment Journal.

This final step closes the loop. The true performance of the model – which includes complex dynamics like delayed user feedback loops that offline proxies missed – is now available to the Offline Agent. This allows the system to update its internal reasoning: if a model had great offline performance but poor live results, the Offline Agent incorporates this negative signal into its future context to avoid similar pitfalls.

More significantly, by automating the repetitive technical execution of model development, the framework shifts the paradigm from human-bottlenecked manual setup to high-velocity evolutionary loops. In this current methodology, human engineers are only required to perform the high-level step of presenting the initial research idea and the final step of reviewing experiment metrics.

5 Deployment and Results

The self-evolving recommendation system has been deployed across several critical surfaces on YouTube. We present a comprehensive evaluation comparing our autonomous dual-agent system against human-engineered baselines.

5.1 Experimental Setup

We evaluate the efficacy of our framework in two stages, aligning with the dual-agent methodology we established. The first stage is offline validation via the Inner Loop, showing that LLM agents are capable of finding candidates that minimize loss or exhibit high correlation with key signals. The second stage is online A/B experimentation via the Outer Loop, showing that candidates reaching this stage significantly improving north star metrics. The underlying production model is an RL fine-tuning model based on a deep neural network to optimize video ranking on YouTube's video watch page, with training typically requiring $\Theta(hours)$.

- **Offline Validation (Inner Loop)** The Offline Agent utilizes proxy signals to filter the potential candidates. For Optimizer and Architecture tasks, it trains hundreds of candidate models asynchronously, filtering by validation loss $\mathcal{L}_{\text{proxy}}$. For Reward tasks, it executes analytical queries over training logs, filtering by feature-label correlation computed from the queries.
- **Online A/B Testing (Outer Loop)** Surviving candidates are promoted to the Online Agent, which orchestrates live A/B tests on a slice of YouTube production traffic.
- **Success Metrics** Final performance is judged against a hierarchy of north star business metrics $\mathcal{M}$.

5.2 Evaluation of the Optimizer and Architecture Tasks: Loss Optimization

The first phase of deployment focused on improving the optimizer and architecture, which both seek to minimize offline proxy loss $\mathcal{L}_{\text{proxy}}$. The Offline Agent identified the below refinements, which yielded significant improvements and outperformed established human-designed baselines when promoted to the live environment (Table 1).

5.2.1 Algorithmic Discovery: Evolving the Optimizer. Traditionally, optimizer configurations remained static due to the high engineering cost of tuning. The agent autonomously identified that switching from the legacy Adagrad optimizer to RMSprop – with a specific learning rate, decay rate, and momentum – resulted in a statistically significant drop in validation loss. In live traffic, both YouTube-level and surface-level business metrics showed significant increase.

5.2.2 System Optimization: Training Efficiency. Beyond model quality, the agent also learned to optimize for system efficiency. By iteratively adjusting batch sizes, training epochs, and optimizer hyperparameters, the agent achieved reductions – first by 4× then by 2× – in training latency without degrading convergence. In total, training time as well as the capacity of the Inner Loop improved by 8× without sacrificing business metrics, proving that the agent can trade off cost and quality.

5.2.3 Structural Discovery: Gated Path Architectures and Activation Refinement. The Architecture persona explored hundreds of potential solutions – ranging from attention mechanisms to Mixture-of-Experts (MoE) – to optimize the topology of the auxiliary towers used for query embedding generation. The agent proposed a Gating Path architecture similar to Gated Linear Units (GLU) [24], which introduced a multiplicative gate to the query embeddings, allowing

Table 1: Agent Tasks vs Discovered Improvements vs Online Metrics

Task	Discovery	YouTube-level metric	Surface-level metric
Optimizer	Transition to RMSprop	+0.06%*	+0.12%*
Optimizer	Training Efficiency (4x Improvement)	−0.01%	+0.06%
Optimizer	Training Efficiency (2x Improvement)	+0.01%	+0.09%*
Architecture	Gated Path (GLU)	+0.06%*	+0.14%*
Architecture	Activation Refinement	−0.02%	+0.12%*
Reward	Multi-Objective Synthesis	+0.03%*	+0.13%*

* denotes results that are statistically significant at the 95% confidence level.

the model to dynamically suppress noise based on context. This innovation yielded some of the most robust gains in our deployment. In a follow-up deployment, the agent further refined this architecture by moving from standard sigmoid gates to GELU activations combined with layer normalization, showing that the agent is capable of both exploring innovative structures and exploiting and fine-tuning structures that it believes to be superior.

5.3 Evaluation of the Reward Task: Semantic Alignment via Signal Correlation

Unlike the Optimizer and Architecture tasks which minimize a clear loss proxy, the Reward improvement task must balance conflicting business objectives while capturing nuanced user behaviors. By leveraging its data analysis tools to identify high-correlation signals in training logs, the agent was able to discover improved reward logic that effectively bridges the gap between training labels and long-term user satisfaction (Table 1).

5.3.1 Semantic Discovery: Multi-Objective Reward Synthesis. Leveraging its tool use and iterative analysis of user interaction patterns, the agent synthesized a composite reward function comprising three novel components:

(1) **Active Engagement** Uses a signal that indicates whether the user is actively engaging with content on the site.
(2) **User-Channel Relationship** Uses a signal that measures user's affinity for the channel.
(3) **Video Quality** Incorporates a new video-level signal related to video quality.

This composite reward function, synthesized entirely by the agent, significantly outperformed the human-engineered baseline. This is a remarkable feat, particularly given the historical difficulty of manual reward engineering. Human researchers often struggle to pinpoint critical semantic bottlenecks within the massive search space of interaction signals, frequently resulting in months of iteration in suboptimal regions of the potential solution space. This underscores the agent's unique ability to perform high-level semantic reasoning – redefining the business logic of success – in ways that traditional optimization processes cannot achieve.

5.4 LLM Performance and Ablation Studies

To understand the drivers of the system's performance, we conducted a series of ablation studies focusing on model selection, persona framing, and context management. These benchmarks highlight the sensitivity of the discovery process to the underlying LLM's reasoning capabilities and the quality of prompt grounding. We consider the task of improving the optimizer and its hyperparameters, using the following variants:

(1) **opt_2p5 (Baseline)** Uses Gemini 2.5 Pro with an expert MLE persona framing, and the full history of past configurations and metrics sorted by offline loss.
(2) **opt_flash** Uses Gemini 2.5 Flash instead of Pro.
(3) **opt_no_role** Ablates the expert MLE persona framing.
(4) **opt_no_sort** Provides the full history of past metrics but ordered by timestamp instead of loss.
(5) **opt_top_1 / opt_top_5** Limits the history to only the top 1 and 5, respectively, sorted by offline loss.
(6) **opt_no_context** Provides no history of past configurations or metrics.

Results are averaged over 6 independent runs exploring 70 ideas each, reported as normalized z-scores of the loss where lower (more negative) values indicate superior performance (Figure 2).

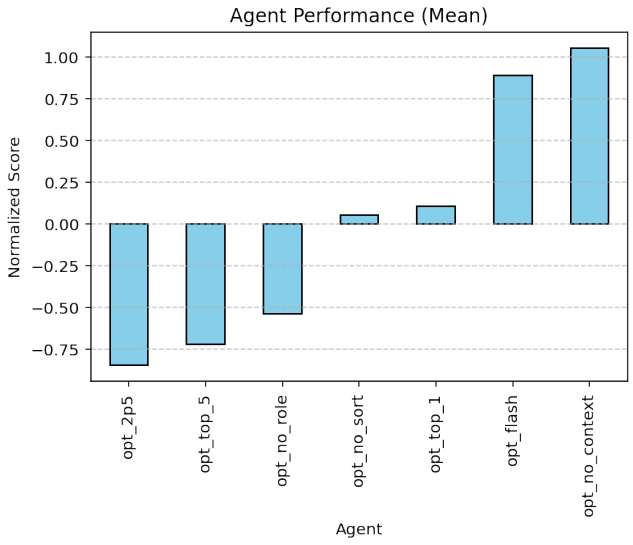

Figure 2: Agent Performance (Normalized $\mathcal{L}_{\text{proxy}}$) across Different Model Sizes and Context Engineering Strategies for Improving Optimizer. The plot shows the mean z-score of the loss. Lower scores indicate superior performance.

Table 2: Experimental Velocity vs Type of Workflow

Metric	Human Workflow	Agent Workflow
Exp. Throughput	$\Theta(1) - \Theta(10)$ / week	$\Theta(100)$ / week
Eng. Cost per Exp	$\Theta(1) - \Theta(10)$ hours / week	0 hours / week

5.4.1 Impact of Model Size and Reasoning. We evaluated the effectiveness of different model choices under the Gemini family. As shown in Figure 2, a larger model with advanced reasoning capabilities significantly outperforms a smaller variant. Specifically, Gemini 2.5 Pro consistently achieves lower loss compared to Gemini 2.5 Flash. This confirms that the reasoning required for algorithmic discovery benefits from the increased parameter count and enhanced "deep thinking" capabilities of the larger model class.

5.4.2 Persona and Context Length. We evaluated the impact of the expert MLE persona framing by comparing it against an agent that lacked the expert identity. This comparison confirms that expert framing significantly influences the relevance and depth of proposed configurations, making it critical for model quality. Similarly, context engineering plays a vital role: providing the full, sorted history from Experiment Journal is better than no history, restricted top-k, or unsorted history. As seen in Figure 2, these results suggest that a comprehensive and ranked distribution of past outcomes is essential for effective iterative discovery.

5.5 Efficiency and The Velocity Dividend

By decoupling the high-frequency offline discovery from the low-frequency online validation, we have removed the human engineer from the critical path of experimentation. The "Idea-to-Data" cycle – the latency from hypothesis generation to experimental results – is significantly cheaper and faster. This velocity dividend now allows the team to produce far more launches than previously possible, as summarized in Table 2 where we compare the experimental velocity between the status quo of manual engineering and our newly proposed agentic engineering.

5.6 Lessons Learned

The deployment of an autonomous, self-evolving system provided several critical insights into the future of recommendation system engineering, ranging from the practical considerations essential for production stability (**L1**-**L3**) to the transformative reasoning power unlocked by an agentic system (**L4**-**L5**).

- **L1: Delta-based vs. Full Configuration Generation** The validity of proposals was significantly enhanced when the agent was tasked with generating a delta against the production file rather than the entire configuration file. Requesting the complete configuration often led to hallucinations where the model would omit essential but unchanged parameters or introduce syntax errors.
- **L2: Enforcing Diversity via Prompt Tuning** Absent explicit instructions, the agent exhibited a strong bias toward safe, incremental changes, effectively collapsing into a mode of minor hyperparameter tuning (e.g., proposing "learning rate 0.1" followed immediately by "learning rate 0.11"). To counteract this, it was critical to prompt the agent to "balance exploration, exploitation, and innovation", forcing it to attempt the leaps necessary for significant gains.
- **L3: The Cold Start Problem** The agent struggles when the Experiment Journal is empty. Without past trials, it tends to propose generic textbook improvements. The system improves markedly after a warm start period, allowing the agent to ground its hypotheses in past learnings.
- **L4: Semantic Reasoning vs. Numerical Tuning** While traditional AutoML excels at fine-tuning scalars like learning rates, our results show that the highest leverage in mature systems comes from structural and semantic mutations. For example, the Reward persona's ability to redefine the business logic of success provided innovations that purely numerical tuning could never achieve.
- **L5: Generalizability Across Recommendation Surfaces** A critical question for any autonomous framework is transferability: can the agent adapt to new environments without re-engineering? We deployed the same dual-agent architecture to a different YouTube recommendation surface with a completely different feature schema, training dataset, and model configuration. Despite these differences, the agents successfully adapted to the new context within the first few iterations, generated valid hypotheses, and increased north star metrics. This confirms that our framework optimizes the process of discovery rather than memorizing a specific dataset, suggesting strong potential for generalization across the broader family of recommendation systems.

6 Conclusion

In this paper, we present a comprehensive framework leveraging Large Language Models (LLMs) for a self-evolving recommendation system, successfully deploying it to scale on the world's largest video delivery platform. By decoupling the discovery process into a fast Offline Agent (driven by cheap proxy signals) and a reliable but slow Online Agent (driven by delayed north star business metrics), we have established a new paradigm for industrial machine learning that overcomes the limitations of traditional workflows. Our extensive deployment results illustrate critical contributions of our work to the field of automated machine learning. We showed that LLMs, when grounded with the appropriate context and tools, are capable of structural and semantic innovation in recommendation system. And by automating the repetitive mechanics of code generation, compilation, and experiment orchestration, we noticeably compressed the "Idea-to-Data" cycle. This order-of-magnitude increase in experimental throughput allows the system to explore the long tail of the configuration space that human engineers simply do not have the bandwidth to investigate.

Looking forward, we envision a shift in the role of the Machine Learning Engineer (MLE). As a self-evolving recommendation system executes on modeling improvements, the human engineer moves focus to defining the strategic guardrails, ethical constraints, and the long-term vision of the system. We believe our work represents a foundational step toward that future, removing human cognitive bandwidth as a bottleneck in scientific discovery in recommendation systems.

References

[1] M. Mehdi Afsar, Trafford Crump, and Behrouz Far. 2022. Reinforcement learning based recommender systems: A survey. arXiv:2101.06286 [cs.IR] https://arxiv.org/abs/2101.06286

[2] Bernd Bischl, Martin Binder, Michel Lang, Tobias Pielok, Jakob Richter, Stefan Coors, Janek Thomas, Theresa Ullmann, Marc Becker, Anne-Laure Boulesteix, Difan Deng, and Marius Lindauer. 2021. Hyperparameter Optimization: Foundations, Algorithms, Best Practices and Open Challenges. arXiv:2107.05847 [stat.ML] https://arxiv.org/abs/2107.05847

[3] Franklin Cardenoso and Wouter Caarls. 2025. Leveraging LLMs for reward function design in reinforcement learning control tasks. arXiv:2511.19355 [cs.LG] https://arxiv.org/abs/2511.19355

[4] Minmin Chen, Alex Beutel, Paul Covington, Sagar Jain, Francois Belletti, and Ed Chi. 2021. Top-K Off-Policy Correction for a REINFORCE Recommender System. arXiv:1812.02353 [cs.LG] https://arxiv.org/abs/1812.02353

[5] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. 2016. Wide & Deep Learning for Recommender Systems. arXiv:1606.07792 [cs.LG] https://arxiv.org/abs/1606.07792

[6] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep Neural Networks for YouTube Recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems* (Boston, Massachusetts, USA) *(RecSys '16)*. Association for Computing Machinery, New York, NY, USA, 191–198. doi:10.1145/2959100.2959190

[7] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. 2019. Neural Architecture Search: A Survey. arXiv:1808.05377 [stat.ML] https://arxiv.org/abs/1808.05377

[8] Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Tobias Springenberg, Manuel Blum, and Frank Hutter. 2015. Efficient and robust automated machine learning. In *Proceedings of the 29th International Conference on Neural Information Processing Systems - Volume 2* (Montreal, Canada) *(NIPS'15)*. MIT Press, Cambridge, MA, USA, 2755–2763.

[9] et al. Gheorghe Comanici. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. arXiv:2507.06261 [cs.CL] https://arxiv.org/abs/2507.06261

[10] Daniel Golovin, Benjamin Solnik, Subhodeep Moitra, Greg Kochanski, John Karro, and D. Sculley. 2017. Google Vizier: A Service for Black-Box Optimization. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Halifax, NS, Canada) *(KDD '17)*. Association for Computing Machinery, New York, NY, USA, 1487–1495. doi:10.1145/3097983.3098043

[11] Dan Hendrycks and Kevin Gimpel. 2023. Gaussian Error Linear Units (GELUs). arXiv:1606.08415 [cs.LG] https://arxiv.org/abs/1606.08415

[12] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. arXiv:2308.00352 [cs.AI] https://arxiv.org/abs/2308.00352

[13] Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. arXiv:1808.09781 [cs.IR] https://arxiv.org/abs/1808.09781

[14] Hanxiao Liu, Karen Simonyan, and Yiming Yang. 2019. DARTS: Differentiable Architecture Search. arXiv:1806.09055 [cs.LG] https://arxiv.org/abs/1806.09055

[15] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. arXiv:2408.06292 [cs.AI] https://arxiv.org/abs/2408.06292

[16] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Eureka: Human-Level Reward Design via Coding Large Language Models. arXiv:2310.12931 [cs.RO] https://arxiv.org/abs/2310.12931

[17] Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, Jinwoo Shin, Sercan Ö. Arık, and Tomas Pfister. 2025. MLE-STAR: Machine Learning Engineering Agent via Search and Targeted Refinement. arXiv:2506.15692 [cs.LG] https://arxiv.org/abs/2506.15692

[18] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G. Azzolini, Dmytro Dzhulgakov, Andrey Mallevich, Ilia Cherniavskii, Yinghai Lu, Raghuraman Krishnamoorthi, Ansha Yu, Volodymyr Kondratenko, Stephanie Pereira, Xianjie Chen, Wenlin Chen, Vijay Rao, Bill Jia, Liang Xiong, and Misha Smelyanskiy. 2019. Deep Learning Recommendation Model for Personalization and Recommendation Systems. arXiv:1906.00091 [cs.IR] https://arxiv.org/abs/1906.00091

[19] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131 [cs.AI] https://arxiv.org/abs/2506.13131

[20] Prajit Ramachandran, Barret Zoph, and Quoc V. Le. 2017. Searching for Activation Functions. arXiv:1710.05941 [cs.NE] https://arxiv.org/abs/1710.05941

[21] Carl Edward Rasmussen and Christopher K. I. Williams. 2005. *Gaussian Processes for Machine Learning*. The MIT Press. doi:10.7551/mitpress/3206.001.0001

[22] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V Le. 2019. Regularized Evolution for Image Classifier Architecture Search. arXiv:1802.01548 [cs.NE] https://arxiv.org/abs/1802.01548

[23] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761 [cs.CL] https://arxiv.org/abs/2302.04761

[24] Noam Shazeer. 2020. GLU Variants Improve Transformer. arXiv:2002.05202 [cs.LG] https://arxiv.org/abs/2002.05202

[25] Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. 2012. Practical Bayesian Optimization of Machine Learning Algorithms. arXiv:1206.2944 [stat.ML] https://arxiv.org/abs/1206.2944

[26] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291 [cs.AI] https://arxiv.org/abs/2305.16291

[27] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. 2017. Deep & Cross Network for Ad Click Predictions. arXiv:1708.05123 [cs.LG] https://arxiv.org/abs/1708.05123

[28] Yuyan Wang, Cheenar Banerjee, Samer Chucri, Fabio Soldo, Sriraj Badam, Ed H. Chi, and Minmin Chen. 2025. Beyond Item Dissimilarities: Diversifying by Intent in Recommender Systems. arXiv:2405.12327 [cs.IR] doi:10.1145/3690624.3709429

[29] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903 [cs.CL] https://arxiv.org/abs/2201.11903

[30] Yi Wu, Daryl Chang, Jennifer She, Zhe Zhao, Li Wei, and Lukasz Heldt. 2024. Learned Ranking Function: From Short-term Behavior Predictions to Long-term User Satisfaction. arXiv:2408.06512 [cs.LG] https://arxiv.org/abs/2408.06512

[31] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large Language Models as Optimizers. arXiv:2309.03409 [cs.LG] https://arxiv.org/abs/2309.03409

[32] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629 [cs.CL] https://arxiv.org/abs/2210.03629

[33] Xiangyu Zhao, Long Xia, Jiliang Tang, and Dawei Yin. 2019. "Deep reinforcement learning for search, recommendation, and online advertising: a survey" by Xiangyu Zhao, Long Xia, Jiliang Tang, and Dawei Yin with Martin Vesely as coordinator. *ACM SIGWEB Newsletter* 2019, Spring (July 2019), 1–15. doi:10.1145/3320496.3320500

[34] Guorui Zhou, Chengru Song, Xiaoqiang Zhu, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep Interest Network for Click-Through Rate Prediction. arXiv:1706.06978 [stat.ML] https://arxiv.org/abs/1706.06978

[35] Marc-André Zöller and Marco F. Huber. 2021. Benchmark and Survey of Automated Machine Learning Frameworks. arXiv:1904.12054 [cs.LG] https://arxiv.org/abs/1904.12054

A LLM Prompt

We present the prompt template utilized by the Offline Agent to generate model improvements.

 # PERSONA
You are a brilliant and innovative machine learning scientist with excellent programming and analytical skills. You want to improve the model and you have deep expertise in {AGENT_SPECIALIZATION}.

 # TASK
Propose changes in {AGENT_TASK} to the model, with the following goals:
- Balance exploration, exploitation, and innovation: You should make X, Y, and Z proposals, respectively, in the three categories.
- Aim to maximize the following online metrics, in order of importance: Metric#1, Metric#2, ..

 # (OPTIONAL) SPECIAL INSTRUCTIONS
..

 # GUARDRAILS
Maintain system safety by enforcing:
- Keep Metric#3 $\leq$ +1%
- ..

 # CONTEXT
The model currently has the following configuration: *[BASELINE CONFIGURATION]*
(Optional) The schema definition for {SCHEMA_NAME} is the following: {SCHEMA_DEFINITION}
Below is the history of past online and offline experiment results: *[EXPERIMENT JOURNAL]*

 # OUTPUT FORMAT
Think step-by-step and double-check syntax. Output each proposal with exactly two fields:
- "explanation", briefly describing what change this is and why it is potentially useful
- "diff", the change against the model's current configuration

 # EXAMPLE PROPOSAL
{AGENT_EXAMPLE}

Figure 3: LLM Prompt. {..} are populated based on the agent's specialized persona (Optimizer, Architecture, or Reward), and [..] are populated from shared context.